

VIDEO STREAMING PLATFORM

PROJECT REPORT

Submitted to

CodTech IT Solutions

Submitted By

Bammidi Ramya Jyothi

Intern ID: CITS4882

Project Name: Video Streaming Platform

Project Scope: Full Stack Web Development

Duration: 8 Weeks

CERTIFICATE

This is to certify that Bammidi Ramya Jyothi, Intern ID CITS4882, has successfully completed the project titled “Video Streaming Platform” as part of the Full Stack Web Development Internship at CodTech IT Solutions.

The project demonstrates the practical implementation of modern web development concepts including frontend development, backend API development, database integration, responsive UI design, routing, and application architecture.

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to CodTech IT Solutions for providing me with the opportunity to work on the Video Streaming Platform project as part of my internship.

This project helped me gain practical knowledge of full-stack web development and provided experience in designing user interfaces, developing APIs, connecting frontend and backend applications, and organizing a complete web application.

I would also like to thank the mentors and learning resources that supported me throughout the development of this project.

TABLE OF CONTENTS

1. Abstract
2. Introduction
3. Project Information
4. Objectives
5. Project Description
6. Technology Stack
7. System Architecture

8. Features
9. Project Structure
10. Installation and Execution
11. Application Workflow
12. Database and API Integration
13. Project Screenshots
14. Challenges Faced
15. Learning Outcomes
16. Future Enhancements
17. Conclusion
18. References

1. ABSTRACT

The Video Streaming Platform is a modern web application developed to provide users with an interactive platform for browsing and watching video content.

The project demonstrates the development of a full-stack application using modern web technologies. The frontend provides a responsive and user-friendly interface, while the backend handles application logic, API requests, and communication with the database.

The platform demonstrates important concepts such as component-based development, client-server communication, REST APIs, database integration, routing, responsive design, and video content management.

2. INTRODUCTION

Video streaming has become one of the most widely used forms of digital content consumption. Modern streaming platforms need to provide users with a simple interface for discovering, browsing, and watching content.

The purpose of this project is to develop a Video Streaming Platform that demonstrates how a modern full-stack web application can be designed and implemented.

The application provides an organized interface for accessing video content while demonstrating frontend and backend integration. It also provides practical experience in handling application data, APIs, navigation, and responsive layouts.

3. PROJECT INFORMATION

Field	Details
Intern ID	CITS4882

Field	Details
Full Name	Bammidi Ramya Jyothi
Project Name	Video Streaming Platform
Project Scope	Full Stack Web Development
Duration	8 Weeks
Organization	CodTech IT Solutions

4. OBJECTIVES

The main objectives of the project are:

- To develop a modern video streaming web application.
- To create a responsive and user-friendly interface.
- To implement reusable frontend components.
- To implement navigation using React Router.
- To develop backend APIs using Node.js and Express.js.
- To integrate a MongoDB database.
- To establish communication between frontend and backend.
- To provide an organized video browsing experience.
- To gain practical experience with Git and GitHub.

5. PROJECT DESCRIPTION

The Video Streaming Platform is a full-stack web application designed to provide users with an engaging video browsing and streaming experience.

The application provides a frontend interface through which users can access available video content and navigate through different sections of the platform.

The frontend is developed using React.js, while the backend is implemented using Node.js and Express.js. MongoDB is used for database management, and Axios is used for communication between the frontend and backend.

The application follows a component-based architecture that makes the project easier to maintain and extend.

6. TECHNOLOGY STACK

6.1 Frontend

React.js

React.js is used to develop the interactive user interface using reusable components.

Vite

Vite is used as the frontend development and build tool.

JavaScript

JavaScript is used to implement application logic and interactive functionality.

HTML5

HTML5 is used to structure the application.

CSS3

CSS3 is used to create the visual design and responsive layouts.

Axios

Axios is used for making HTTP requests and communicating with backend APIs.

React Router DOM

React Router DOM is used to manage navigation between different pages.

6.2 Backend

Node.js

Node.js provides the runtime environment for the backend application.

Express.js

Express.js is used to create backend APIs and handle HTTP requests.

6.3 Database

MongoDB

MongoDB is used to store application data.

Mongoose

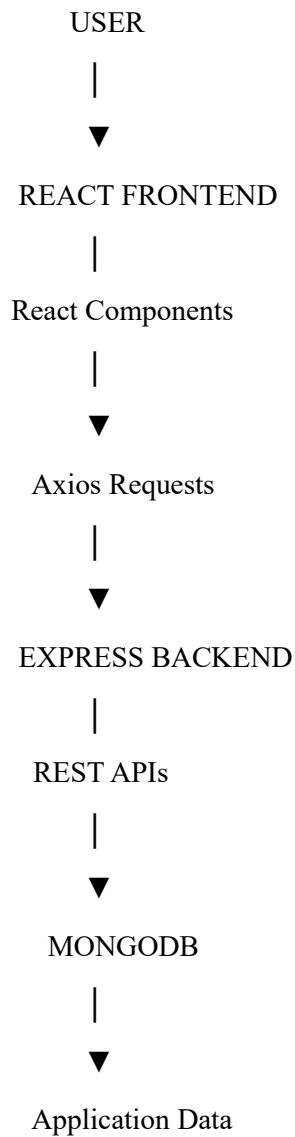
Mongoose is used to interact with MongoDB and define database models.

6.4 Version Control

- Git
- GitHub

7. SYSTEM ARCHITECTURE

The application follows a client-server architecture.



The frontend communicates with the backend through API requests. The backend processes requests and interacts with MongoDB to store and retrieve application data.

8. FEATURES

8.1 Video Browsing

Users can browse available video content through the application's interface.

8.2 Video Playback

The platform provides a video viewing interface for playing available content.

8.3 Video Details

Users can access information related to selected videos.

8.4 Search

The platform can provide search functionality to help users find relevant video content.

8.5 Dashboard

The dashboard acts as the main interface from which users can access available sections and content.

8.6 Responsive Interface

The application is designed to work across different screen sizes.

8.7 API Integration

The frontend communicates with the backend through APIs.

8.8 Database Integration

MongoDB is used to store and manage application data.

8.9 Component-Based Architecture

Reusable React components are used to maintain an organized and scalable codebase.

9. PROJECT STRUCTURE

video-streaming-platform/

```
|
|
| — frontend/
|   |
|   | — src/
|   |   |
|   |   | — components/
|   |   |   |
|   |   |   | — pages/
|   |   |   |   |
|   |   |   |   | — assets/
|   |   |   |   |   |
|   |   |   |   |   | — App.jsx
|   |   |   |   |   |   |
|   |   |   |   |   |   | — main.jsx
|   |   |
|   |   |
|   |   | — public/
|   |   | — package.json
|   |   | — vite.config.js
|   |
|   |
| — backend/
|   |
|   | — controllers/
|   | — models/
|   | — routes/
|   | — middleware/
|   | — server.js
|   | — package.json
```

```
|
|— screenshots/
|
|— documentation/
|   └─ Project_Report.pdf
|
|— README.md
└─ .gitignore
```

10. INSTALLATION AND EXECUTION

Step 1: Clone the Repository

```
git clone https://github.com/ramyajyothi-bammidi/video-streaming-platform.git
```

Step 2: Navigate to the Project

```
cd video-streaming-platform
```

Step 3: Install Frontend Dependencies

```
cd frontend
```

```
npm install
```

Step 4: Install Backend Dependencies

```
cd ../backend
```

```
npm install
```

Step 5: Start the Backend

```
npm run dev
```

The backend runs according to the port configured in the project.

Step 6: Start the Frontend

Open another terminal:

```
cd frontend
```

```
npm run dev
```

The frontend can normally be accessed through:

```
http://localhost:5173
```

11. APPLICATION WORKFLOW

The application follows the following workflow:

User



Application Home Page



Browse Videos



Search / Select Video



Video Details



Video Player



Watch Content

The user starts from the main application interface and can browse available videos. After selecting a video, the user can view its details and access the video player.

12. DATABASE AND API INTEGRATION

The application uses a backend API to manage communication between the frontend and database.

The basic architecture is:

React Frontend



| HTTP Request



Express.js API



| Database Query



MongoDB

|

| Response



Express.js

|



React Frontend

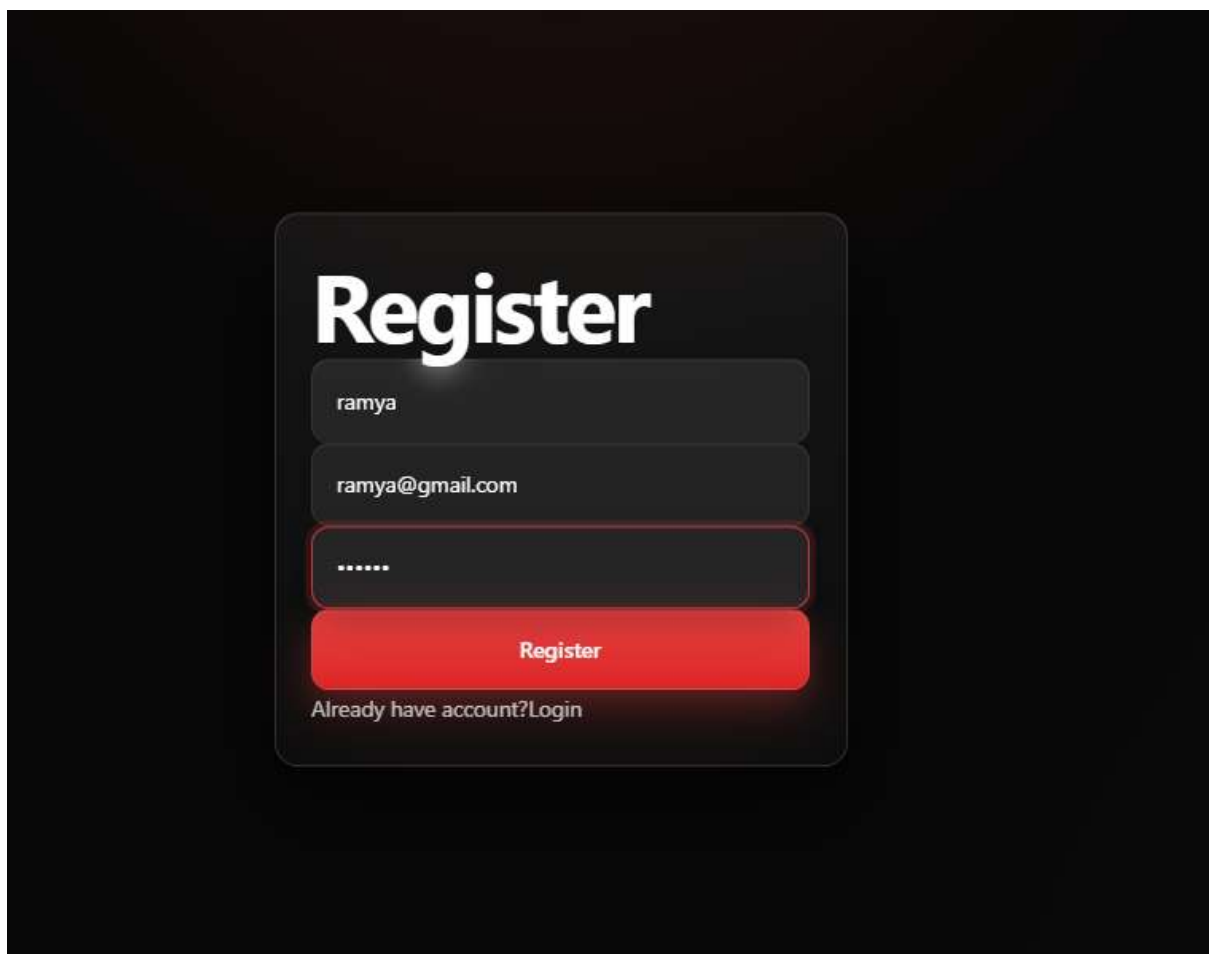
Axios is used on the frontend to send HTTP requests to the backend.

MongoDB provides persistent storage for application-related information, while Mongoose helps manage database models and queries.

12. PROJECT SCREENSHOTS

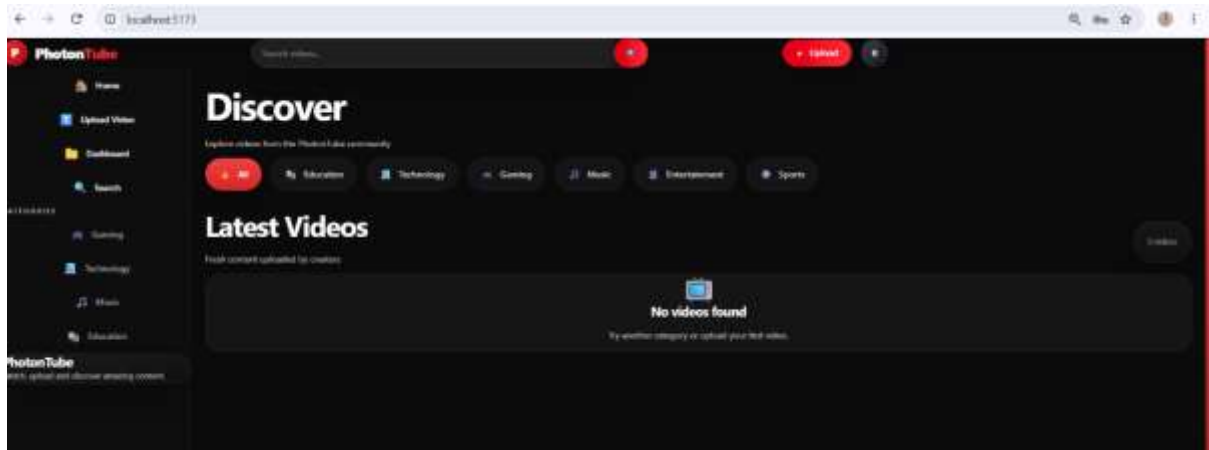
The following screenshots demonstrate the main interfaces of the project.

12.1 Registration Page



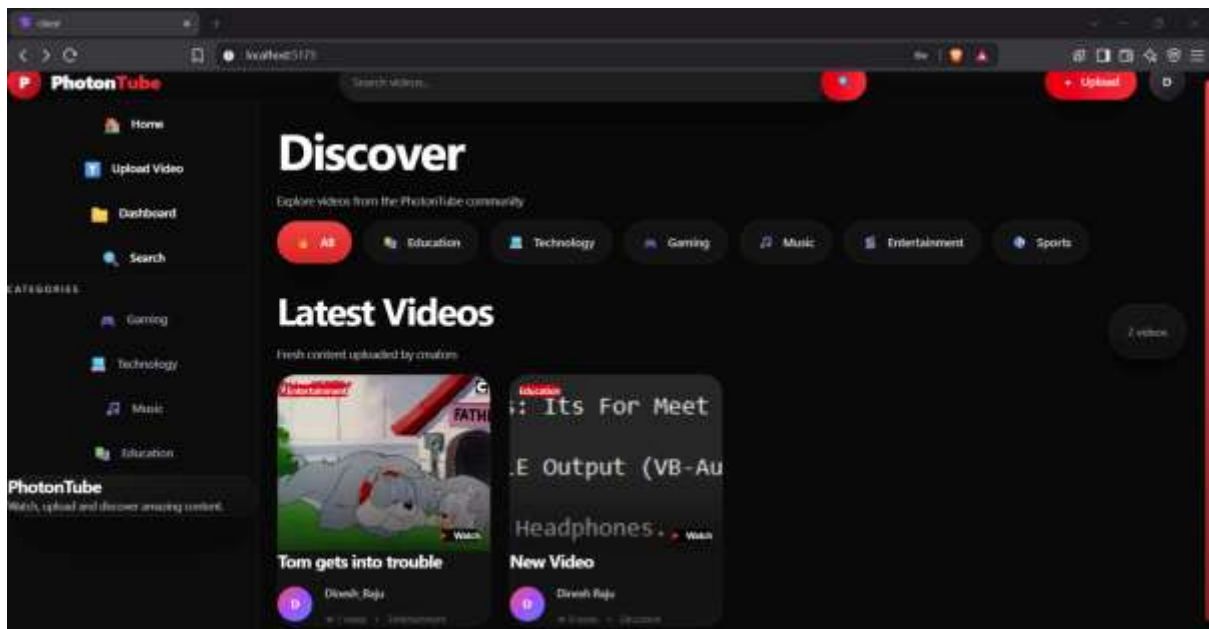
The registration page provides the interface through which users can enter the required information and access the application.

12.2 Dashboard



The dashboard provides the main application interface where users can access different sections and view available content.

12.3 Latest Videos

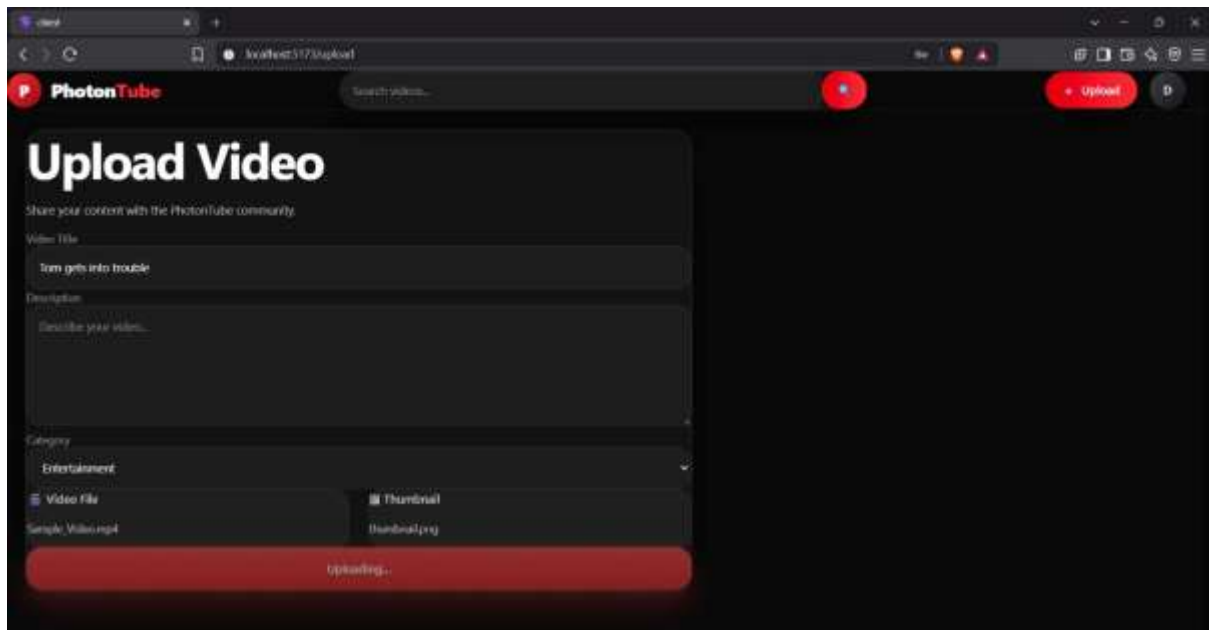


The latest videos section provides users with an interface for viewing available video-related content.

12.4 Upload Page

Screenshot: upload.jpeg

The upload interface provides a section for users to upload content through the application.



14. CHALLENGES FACED

During the development of the project, several challenges were encountered.

Frontend Development

Creating a clean and responsive interface required careful component design and CSS implementation.

Component Management

Organizing different sections into reusable React components required proper project planning.

Backend Integration

Connecting the React frontend with the Express backend required proper API configuration and request handling.

Database Connectivity

Establishing communication between the backend and MongoDB required configuring the database connection and models correctly.

Routing

Implementing navigation between multiple application pages required proper configuration of React Router.

Debugging

Development involved identifying and resolving frontend, backend, routing, dependency, and API-related issues.

These challenges improved practical problem-solving and debugging skills.

15. LEARNING OUTCOMES

The development of this project provided practical experience in:

- React.js
- Component-Based Development
- JavaScript ES6+
- Vite
- React Router
- Axios
- REST API Development
- Node.js
- Express.js
- MongoDB
- Mongoose
- Responsive Web Design
- Frontend and Backend Integration
- Git and GitHub
- Debugging and Troubleshooting

The project helped strengthen understanding of how frontend, backend, APIs, and databases work together in a full-stack application.

16. FUTURE ENHANCEMENTS

The platform can be further enhanced with the following features:

- User Authentication
- User Profiles
- Video Upload
- Like and Dislike System
- Comments
- Watchlist
- Watch History
- Ratings and Reviews
- Search and Advanced Filtering

- Real-Time Notifications
- Dark Mode
- Multiple Video Quality Options
- Continue Watching
- Personalized Video Recommendations
- Mobile Application

These improvements can make the platform more feature-rich and closer to a production-level streaming application.

17. CONCLUSION

The Video Streaming Platform project successfully demonstrates the development of a modern full-stack web application.

The project combines React.js for frontend development, Node.js and Express.js for backend services, and MongoDB for data management. It demonstrates important concepts such as component-based architecture, routing, API integration, database connectivity, and responsive web design.

Through the development of this project, valuable practical experience was gained in designing, developing, integrating, testing, and managing a complete web application.

The project also strengthened knowledge of full-stack development and provided hands-on experience with technologies commonly used in modern web application development.

18. REFERENCES

1. React.js Documentation
2. Vite Documentation
3. React Router Documentation
4. Axios Documentation
5. Node.js Documentation
6. Express.js Documentation
7. MongoDB Documentation
8. Mongoose Documentation
9. MDN Web Docs
10. Git Documentation
11. GitHub Documentation

AUTHOR

Bammidi Ramya Jyothi

Intern ID: CITS4882

Project Name: Video Streaming Platform

Project Scope: Full Stack Web Development

Organization: CodTech IT Solutions

GitHub:

<https://github.com/ramyajyothi-bammidi/video-streaming-platform>

DECLARATION

I hereby declare that the Video Streaming Platform project was developed by me as part of my internship at CodTech IT Solutions. The project was developed for educational and internship purposes using the technologies and concepts described in this report.

Name: Bammidi Ramya Jyothi

Intern ID: CITS4882

Project: Video Streaming Platform

Organization: CodTech IT Solutions